

# Business Problem

Many businesses receive thousands of customer reviews every day. Manually analyzing these reviews is time-consuming and inefficient. The challenge is to automatically determine customer sentiment (Positive, Neutral, or Negative) and extract meaningful insights that help improve products and customer satisfaction.

## Project Goal

Develop a sentiment analysis model using Natural Language Processing (NLP) techniques to classify Amazon customer reviews into Positive, Neutral, and Negative sentiments and generate business insights from customer feedback.

## Objectives

Analyze Amazon customer reviews.

Clean and preprocess text data.

Perform Exploratory Data Analysis (EDA).

Apply NLP techniques.

Classify customer sentiment.

Visualize sentiment distribution.

Provide actionable business recommendations.

## Import Libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
```

```

import seaborn as sns

import nltk
import re
import string

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

from wordcloud import WordCloud

from nltk.sentiment import SentimentIntensityAnalyzer

from sklearn.model_selection import train_test_split

from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

df = pd.read_csv('/content/drive/MyDrive/Reviews.csv')

df.head()

{"type": "dataframe", "variable_name": "df"}

df.tail()

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"Id\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1,\n        \"min\": 568450,\n        \"max\": 568454,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          568451,\n          568454,\n          568452\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"ProductId\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"B003S1WTCU\",\n          \"B001LR2CU2\",\n          \"B001E07N10\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"UserId\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"A3I8AFVPEE8KI5\",\n          \"A3LGQPJCZVL9UC\",\n          \"A121AA1GQV751Z\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"ProfileName\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"R. Sawyer\",\n          \"srfell17\",\n          \"pk007\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  ]\n}"}

```

```

\ "HelpfulnessNumerator\ ",\n          \ "properties\ ": {\n          \ "dtype\ ":
\ "number\ ",\n          \ "std\ ": 0,\n          \ "min\ ": 0,\n
\ "max\ ": 2,\n          \ "num_unique_values\ ": 3,\n          \ "samples\ ":
[\n          0,\n          2,\n          1\n          ],\n
\ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n          }\n
n      },\n      {\n          \ "column\ ": \ "HelpfulnessDenominator\ ",\n
\ "properties\ ": {\n          \ "dtype\ ": \ "number\ ",\n          \ "std\ ":
0,\n          \ "min\ ": 0,\n          \ "max\ ": 2,\n
\ "num_unique_values\ ": 3,\n          \ "samples\ ": [\n          0,\n
2,\n          1\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n      },\n      {\n          \ "column\ ":
\ "Score\ ",\n          \ "properties\ ": {\n          \ "dtype\ ": \ "number\ ",\n
\ "std\ ": 1,\n          \ "min\ ": 2,\n          \ "max\ ": 5,\n
\ "num_unique_values\ ": 2,\n          \ "samples\ ": [\n          2,\n
5\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n      },\n      {\n          \ "column\ ":
\ "Time\ ",\n          \ "properties\ ": {\n          \ "dtype\ ": \ "number\ ",\n
\ "std\ ": 15189380,\n          \ "min\ ": 1299628800,\n          \ "max\ ":
1338422400,\n          \ "num_unique_values\ ": 5,\n          \ "samples\ ":
[\n          1331251200,\n          1338422400\n          ],\n
\ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n          }\n
n      },\n      {\n          \ "column\ ": \ "Summary\ ",\n          \ "properties\ ":
{\n          \ "dtype\ ": \ "string\ ",\n          \ "num_unique_values\ ": 5,\n
\ "samples\ ": [\n          \ "disappointed\ ",\n          \ "Great
Honey\ "\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n      },\n      {\n          \ "column\ ":
\ "Text\ ",\n          \ "properties\ ": {\n          \ "dtype\ ": \ "string\ ",\n
\ "num_unique_values\ ": 5,\n          \ "samples\ ": [\n          \ "I'm
disappointed with the flavor. The chocolate notes are especially weak.
Milk thickens it but the flavor still disappoints. This was worth a
try but I'll never buy again. I will use what's left, which will be
gone in no time thanks to the small cans.\",\n          \ "I am very
satisfied ,product is as advertised, I use it on cereal, with raw
vinegar, and as a general sweetner.\",\n          ],\n
\ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n          }\n
n      }\n  ]\n}","type":"dataframe"}

```

df.shape

```
(568454, 10)
```

df.describe()

```

{"summary":{"\n  \ "name\ ": \ "df\ ",\n  \ "rows\ ": 8,\n  \ "fields\ ": [\n
{\n          \ "column\ ": \ "Id\ ",\n          \ "properties\ ": {\n
\ "dtype\ ": \ "number\ ",\n          \ "std\ ": 204873.2671250089,\n
\ "min\ ": 1.0,\n          \ "max\ ": 568454.0,\n
\ "num_unique_values\ ": 6,\n          \ "samples\ ": [\n
568454.0,\n          284227.5,\n          426340.75\n          ],\n
\ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n          }\n
n      ]\n}"}

```

```

n    },\n    {\n        \"column\": \"HelpfulnessNumerator\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 200934.75222383515, \n            \"min\": 0.0, \n            \"max\": 568454.0, \n            \"num_unique_values\": 6, \n            \"samples\": [\n                568454.0, \n                1.7438174416927315, \n                866.0\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }\n    },\n    {\n        \"column\": \"HelpfulnessDenominator\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 200931.7961032001, \n            \"min\": 0.0, \n            \"max\": 568454.0, \n            \"num_unique_values\": 7, \n            \"samples\": [\n                568454.0, \n                2.2288100708236724, \n                2.0\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }\n    },\n    {\n        \"column\": \"Score\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 200977.55147984123, \n            \"min\": 1.0, \n            \"max\": 568454.0, \n            \"num_unique_values\": 6, \n            \"samples\": [\n                568454.0, \n                4.183198640523243, \n                5.0\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }\n    },\n    {\n        \"column\": \"Time\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 582571406.0467855, \n            \"min\": 568454.0, \n            \"max\": 1351209600.0, \n            \"num_unique_values\": 8, \n            \"samples\": [\n                1296256604.9024196, \n                1311120000.0, \n                568454.0\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n        }\n    }\n    ],\n    \"type\": \"dataframe\"}

```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 568454 entries, 0 to 568453
```

```
Data columns (total 10 columns):
```

| # | Column                 | Non-Null Count  | Dtype  |
|---|------------------------|-----------------|--------|
| 0 | Id                     | 568454 non-null | int64  |
| 1 | ProductId              | 568454 non-null | object |
| 2 | UserId                 | 568454 non-null | object |
| 3 | ProfileName            | 568428 non-null | object |
| 4 | HelpfulnessNumerator   | 568454 non-null | int64  |
| 5 | HelpfulnessDenominator | 568454 non-null | int64  |
| 6 | Score                  | 568454 non-null | int64  |
| 7 | Time                   | 568454 non-null | int64  |
| 8 | Summary                | 568427 non-null | object |
| 9 | Text                   | 568454 non-null | object |

```
dtypes: int64(5), object(5)
```

```
memory usage: 43.4+ MB
```

```
df.isnull().sum()
```

|           |   |
|-----------|---|
| Id        | 0 |
| ProductId | 0 |

```

UserId          0
ProfileName     26
HelpfulnessNumerator  0
HelpfulnessDenominator  0
Score           0
Time           0
Summary        27
Text           0
dtype: int64

df.duplicated().sum()

np.int64(0)

df.drop_duplicates()

{"type": "dataframe"}

df['Score'].value_counts()

Score
5    363122
4     80655
1     52268
3     42640
2     29769
Name: count, dtype: int64

def sentiment(score):
    if score <=2:
        return "Negative"

    elif score==3:
        return "Neutral"

    else:
        return "Positive"

df["Sentiment"] = df["Score"].apply(sentiment)

df.head()

{"type": "dataframe", "variable_name": "df"}

df.to_csv("Cleaned_Amazon_Reviews.csv", index=False)

import matplotlib.pyplot as plt
import seaborn as sns

# Set style
sns.set_style("whitegrid")

```

```
# Set figure size
plt.rcParams['figure.figsize'] = (10,6)

plt.figure(figsize=(8,5))

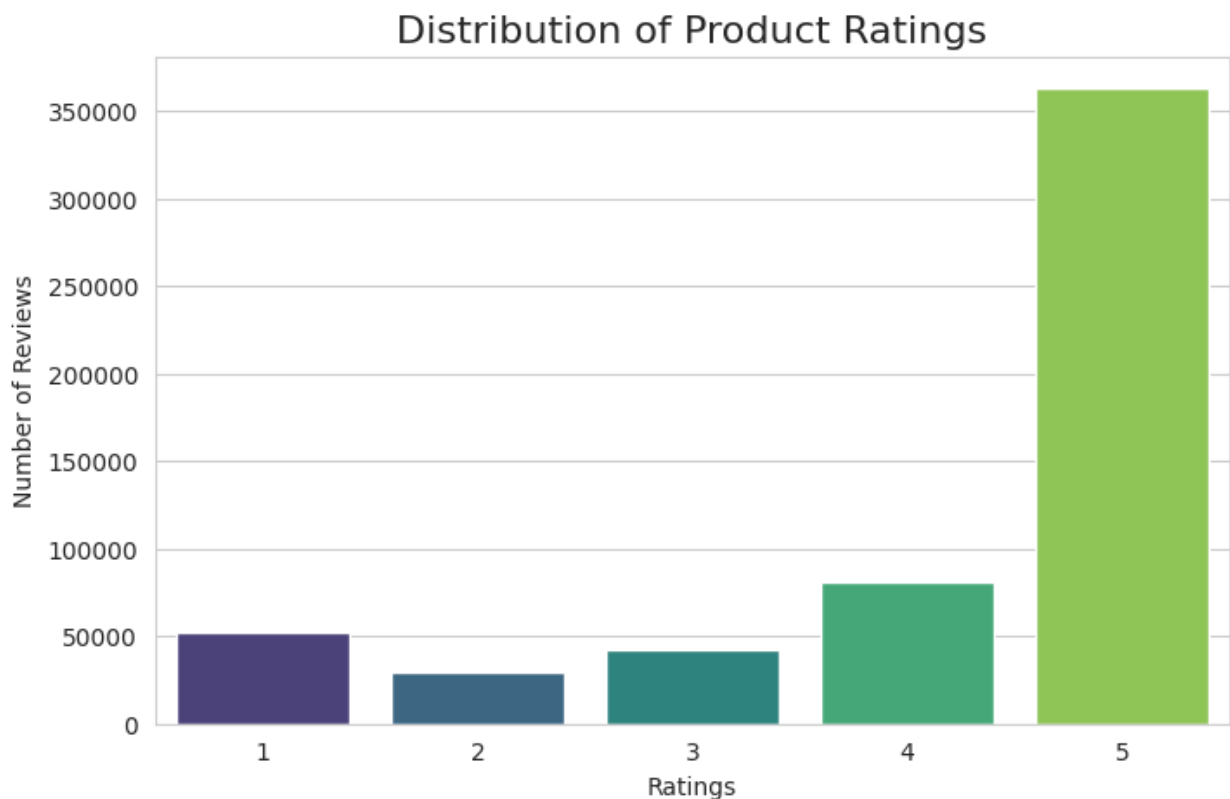
sns.countplot(
    x='Score',
    data=df,
    palette='viridis'
)

plt.title('Distribution of Product Ratings', fontsize=16)
plt.xlabel('Ratings')
plt.ylabel('Number of Reviews')

plt.show()

/tmp/ipykernel_2797/1255228158.py:3: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

sns.countplot(
```



```
plt.figure(figsize=(8,5))
```

```
sns.countplot(  
    x='Sentiment',  
    data=df,  
    palette='Set2'  
)
```

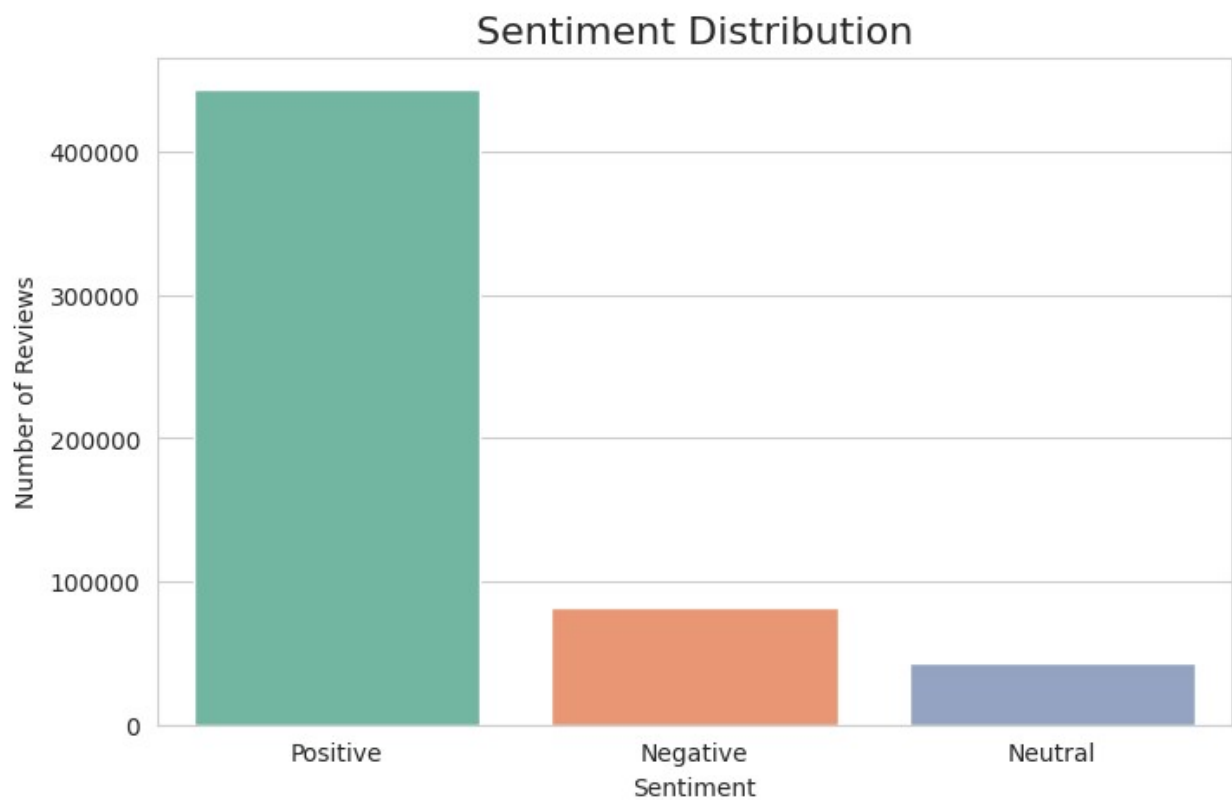
```
plt.title('Sentiment Distribution', fontsize=16)  
plt.xlabel('Sentiment')  
plt.ylabel('Number of Reviews')
```

```
plt.show()
```

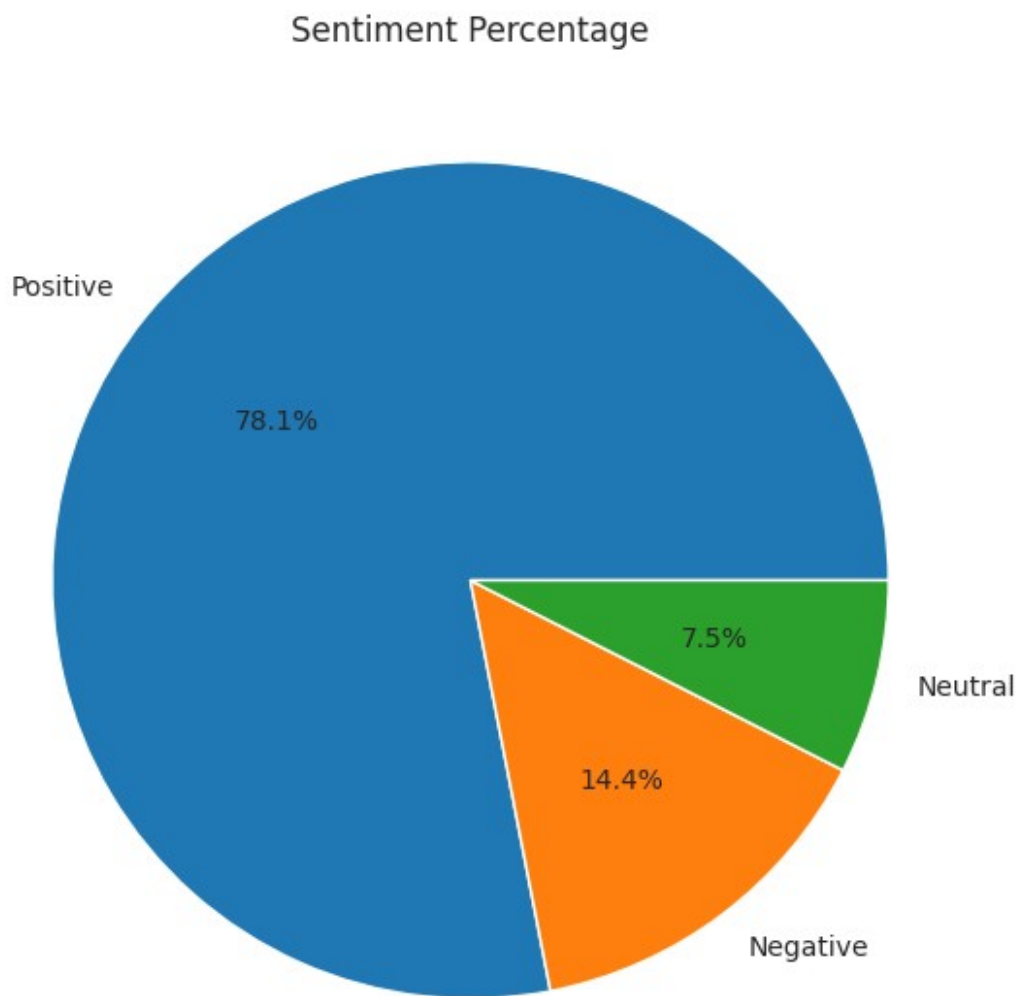
```
/tmp/ipykernel_2797/3657199462.py:3: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be  
removed in v0.14.0. Assign the `x` variable to `hue` and set  
`legend=False` for the same effect.
```

```
sns.countplot(
```



```
plt.figure(figsize=(7,7))
df['Sentiment'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%'
)
plt.title('Sentiment Percentage')
plt.ylabel('')
plt.show()
```



```

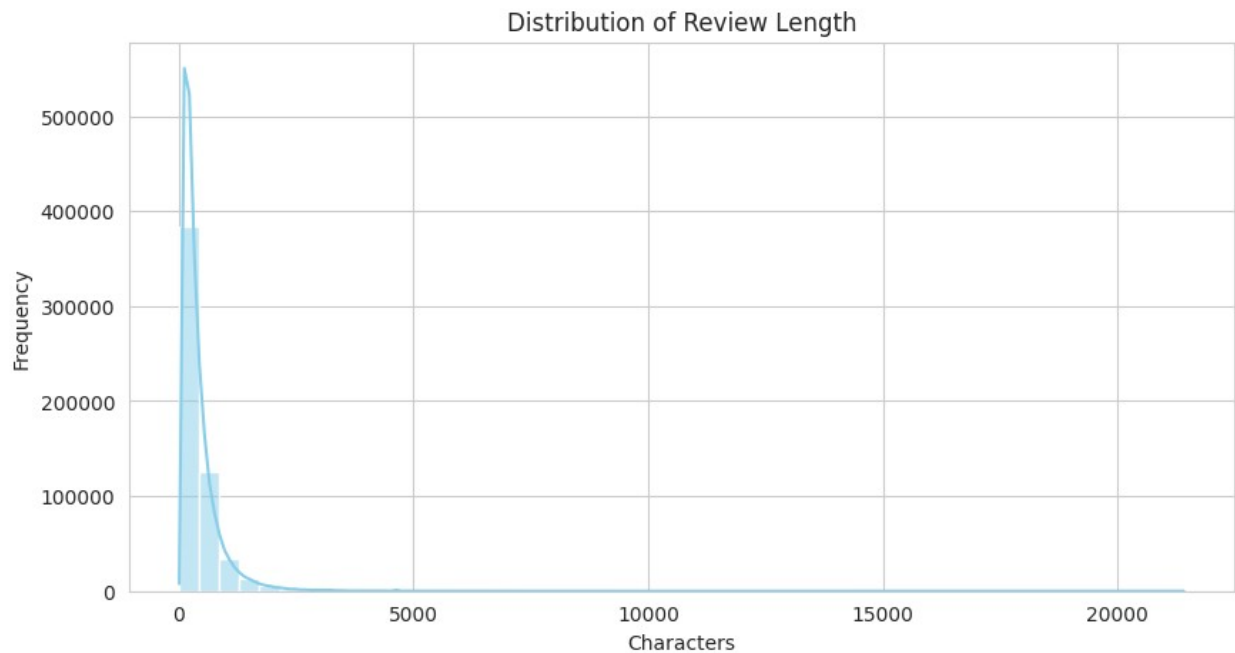
df['Review_Length'] = df['Text'].apply(len)

plt.figure(figsize=(10,5))

sns.histplot(
    df['Review_Length'],
    bins=50,
    kde=True,
    color='skyblue'
)

plt.title('Distribution of Review Length')
plt.xlabel('Characters')
plt.ylabel('Frequency')
plt.show()

```



```

plt.figure(figsize=(8,5))

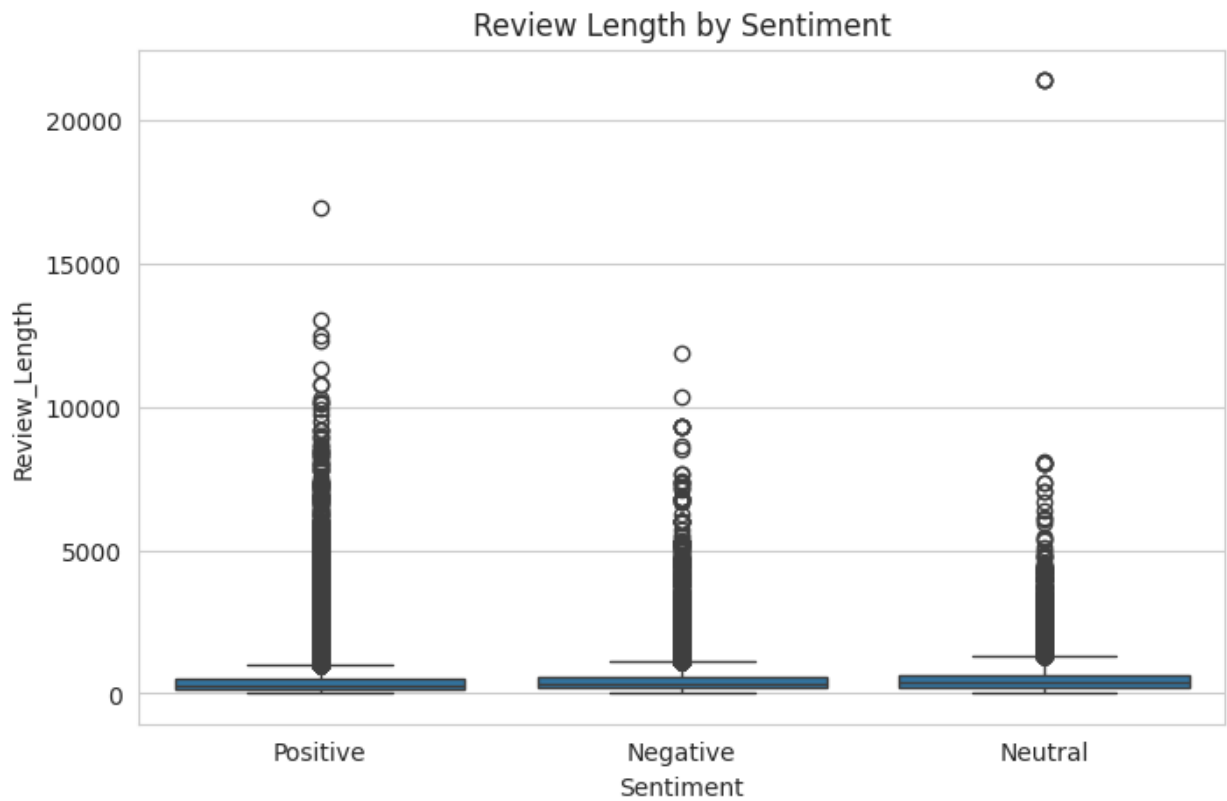
sns.boxplot(
    x='Sentiment',
    y='Review_Length',
    data=df
)

```

```

)
plt.title('Review Length by Sentiment')
plt.show()

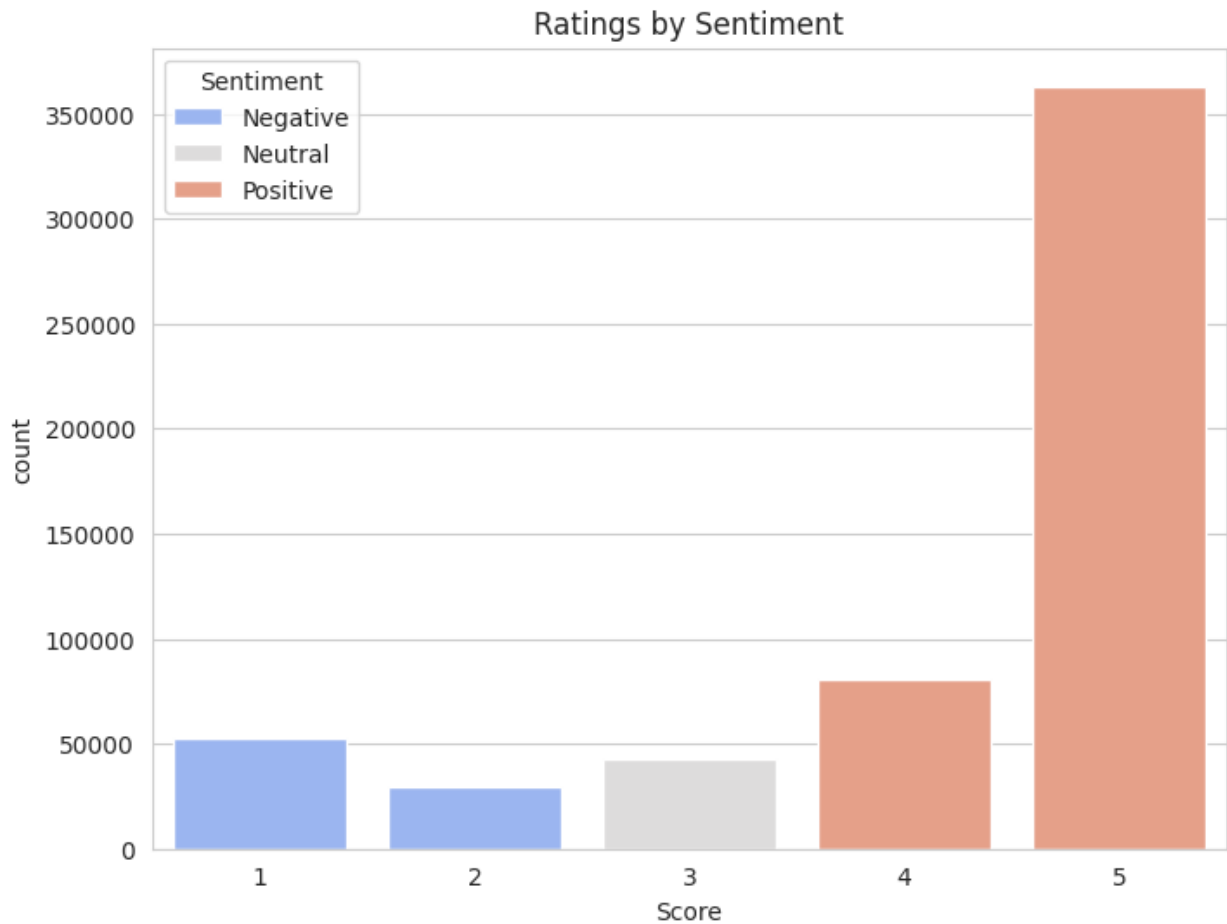
```



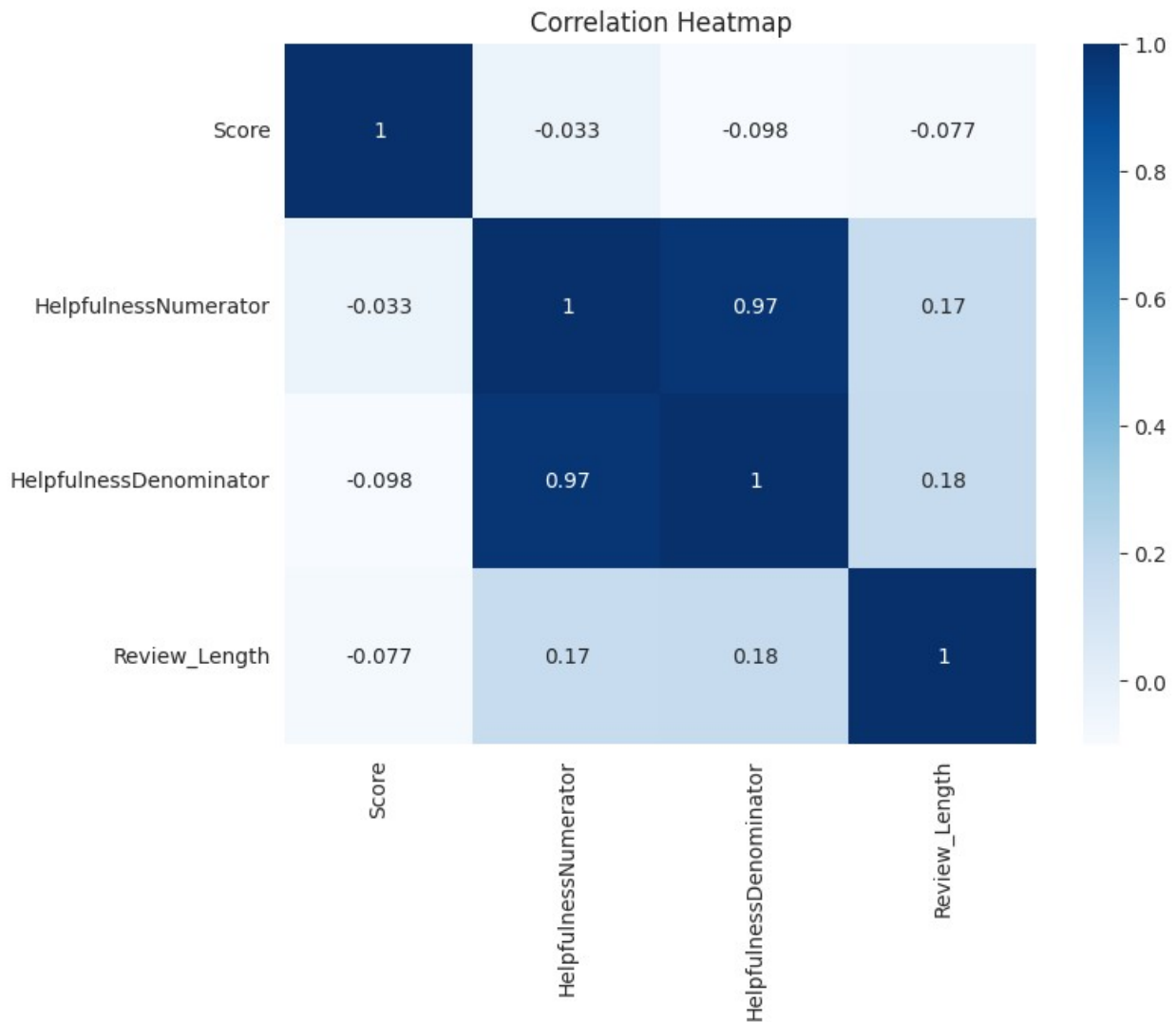
```

plt.figure(figsize=(8,6))
sns.countplot(
    x='Score',
    hue='Sentiment',
    data=df,
    palette='coolwarm'
)
plt.title('Ratings by Sentiment')
plt.show()

```



```
numeric_df = df[['Score',  
                 'HelpfulnessNumerator',  
                 'HelpfulnessDenominator',  
                 'Review_Length']]  
  
plt.figure(figsize=(8,6))  
  
sns.heatmap(  
    numeric_df.corr(),  
    annot=True,  
    cmap='Blues'  
)  
  
plt.title('Correlation Heatmap')  
plt.show()
```



Review length shows a moderate relationship with helpfulness, while ratings have relatively very weak correlations with the other numerical variables

## 1. Analyze text data to classify it as Positive, Negative, or Neutral

```
def sentiment(score):  
    if score >= 4:  
        return "Positive"  
    elif score == 3:  
        return "Neutral"  
    else:
```

```

        return "Negative"

df['Sentiment'] = df['Score'].apply(sentiment)

df[['Score', 'Sentiment']].head()

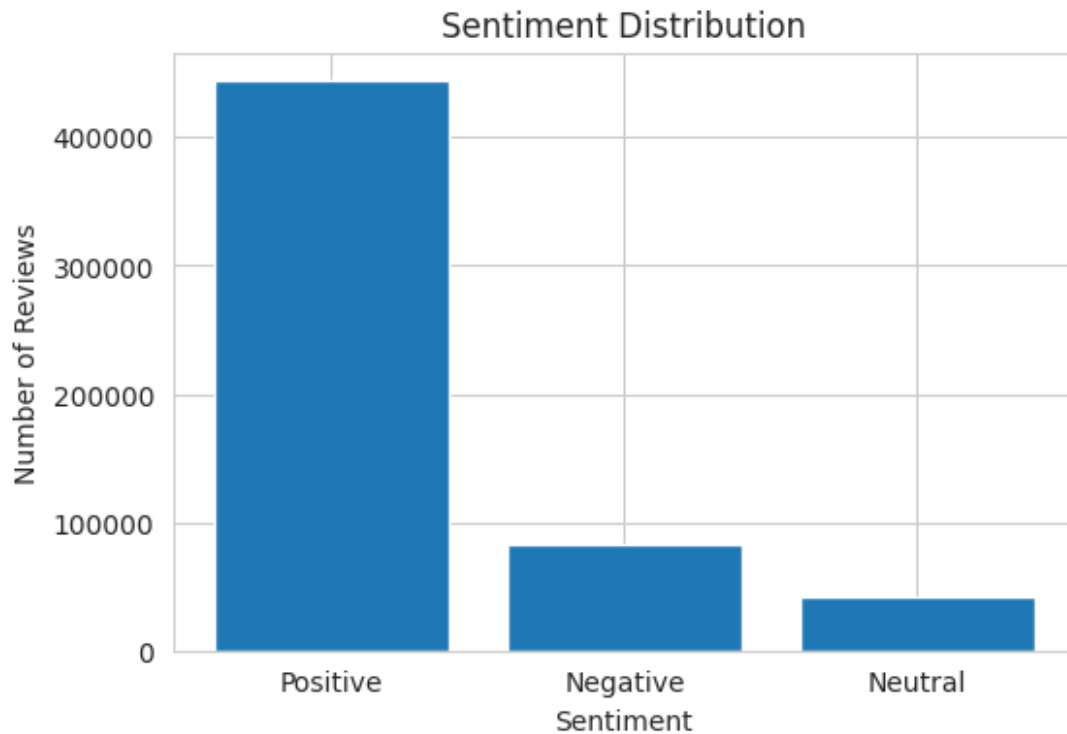
{"summary": "{\n  \"name\": \"df[['Score', 'Sentiment']]\", \n  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"Score\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 1, \n        \"min\": 1, \n        \"max\": 5, \n        \"num_unique_values\": 4, \n        \"samples\": [\n          1, \n          2, \n          5\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      \"column\": \"Sentiment\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 2, \n        \"samples\": [\n          \"Negative\", \n          \"Positive\"\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    ]\n  }, \n  \"type\": \"dataframe\"}

import matplotlib.pyplot as plt

sentiment_count = df['Sentiment'].value_counts()

plt.figure(figsize=(6,4))
plt.bar(sentiment_count.index, sentiment_count.values)
plt.title("Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Number of Reviews")
plt.show()

```



```
import nltk
nltk.download('stopwords')
nltk.download('wordnet')

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!

def preprocess_text(text):

    # Convert to lowercase
    text = text.lower()

    # Remove HTML tags
    text = re.sub(r'<.*?>', '', text)

    # Remove URLs
    text = re.sub(r'http\S+|www\S+', '', text)

    # Remove numbers
```

```

text = re.sub(r'\d+', '', text)

# Remove punctuation
text = text.translate(str.maketrans('', '', string.punctuation))

# Remove extra spaces
text = text.strip()

# Tokenization
words = text.split()

# Remove stopwords
words = [word for word in words if word not in stop_words]

# Lemmatization
words = [lemmatizer.lemmatize(word) for word in words]

# Join words
return " ".join(words)

df['Cleaned_Text'] = df['Text'].astype(str).apply(preprocess_text)

```

```

print("Original Review:\n")
print(df['Text'][0])

```

```

print("\nCleaned Review:\n")
print(df['Cleaned_Text'][0])

```

Original Review:

I have bought several of the Vitality canned dog food products and have found them all to be of good quality. The product looks more like a stew than a processed meat and it smells better. My Labrador is finicky and she appreciates this product better than most.

Cleaned Review:

bought several vitality canned dog food product found good quality product look like stew processed meat smell better labrador finicky appreciates product better

```

df['Cleaned_Text'].isnull().sum()

np.int64(0)

```

## ## 2. Use NLP techniques and lexicons to detect specific emotions

Besides Positive/Negative/Neutral, your task specifically asks for emotion detection.

```
import nltk
nltk.download('vader_lexicon')

[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
True

from nltk.sentiment import SentimentIntensityAnalyzer

sia = SentimentIntensityAnalyzer()

review = df['Text'][0]

scores = sia.polarity_scores(review)

print(scores)

{'neg': 0.0, 'neu': 0.695, 'pos': 0.305, 'compound': 0.9441}

def vader_sentiment(text):
    score = sia.polarity_scores(str(text))

    if score['compound'] >= 0.05:
        return "Positive"
    elif score['compound'] <= -0.05:
        return "Negative"
    else:
        return "Neutral"

df['VADER_Sentiment'] = df['Text'].apply(vader_sentiment)

df[['Text', 'VADER_Sentiment']].head()

{"summary": "{\n  \"name\": \"df[['Text', 'VADER_Sentiment']]\", \n\n  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"Text\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 5, \n        \"samples\": [\n          \"Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized unsalted. Not sure if this was an error or if the vendor intended to represent the product as \\\"Jumbo\\\".\", \n          \"Great taffy at a great price. There was a wide assortment of yummy taffy. Delivery was very quick. If your a taffy lover, this is a deal.\", \n          \"This is a confection that has been around a few centuries. It is a light, pillowy citrus gelatin with nuts - in this case Filberts. And it is cut into tiny squares and then liberally coated with powdered sugar. And it is a tiny mouthful of heaven. Not too chewy, and very flavorful. I highly recommend this yummy treat. If you are familiar with the story of C.S. Lewis' \\\"The Lion, The Witch, and The Wardrobe\\\" - this is the treat that seduces Edmund\" ]\n    ]\n  ]\n}"
```

```
into selling out his Brother and Sisters to the Witch.\n\n    ],\n    \"semantic_type\": \"\",\n    \"description\": \"\"\n  },\n  {\n    \"column\": \"VADER_Sentiment\", \n    \"properties\": {\n      \"dtype\": \"string\", \n      \"num_unique_values\": 3,\n      \"samples\": [\n        \"Positive\", \n        \"Negative\", \n        \"Neutral\" \n      ],\n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }\n  ]\n}","type":"dataframe"}

```

### 3. Apply analysis on Amazon Reviews

```
sentiment_percent = (
    df['VADER_Sentiment']
    .value_counts(normalize=True)
    .mul(100)
    .round(2)
)

print(sentiment_percent)

VADER_Sentiment
Positive      87.43
Negative      10.44
Neutral        2.14
Name: proportion, dtype: float64

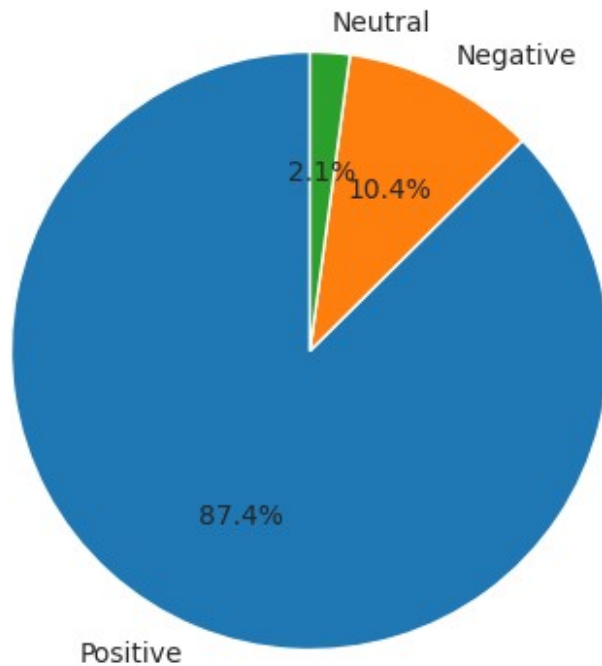
plt.figure(figsize=(7,5))

plt.pie(
    sentiment_percent.values,
    labels=sentiment_percent.index,
    autopct='%1.1f%%',
    startangle=90
)

plt.title("Overall Customer Sentiment")

plt.show()
```

Overall Customer Sentiment



## 4. Understand public opinion and trends

```
df.groupby('VADER_Sentiment')['Score'].mean()
```

```
VADER_Sentiment
Negative      2.588085
Neutral      3.389081
Positive     4.393048
Name: Score, dtype: float64
```

```
df['VADER_Sentiment'].value_counts()
```

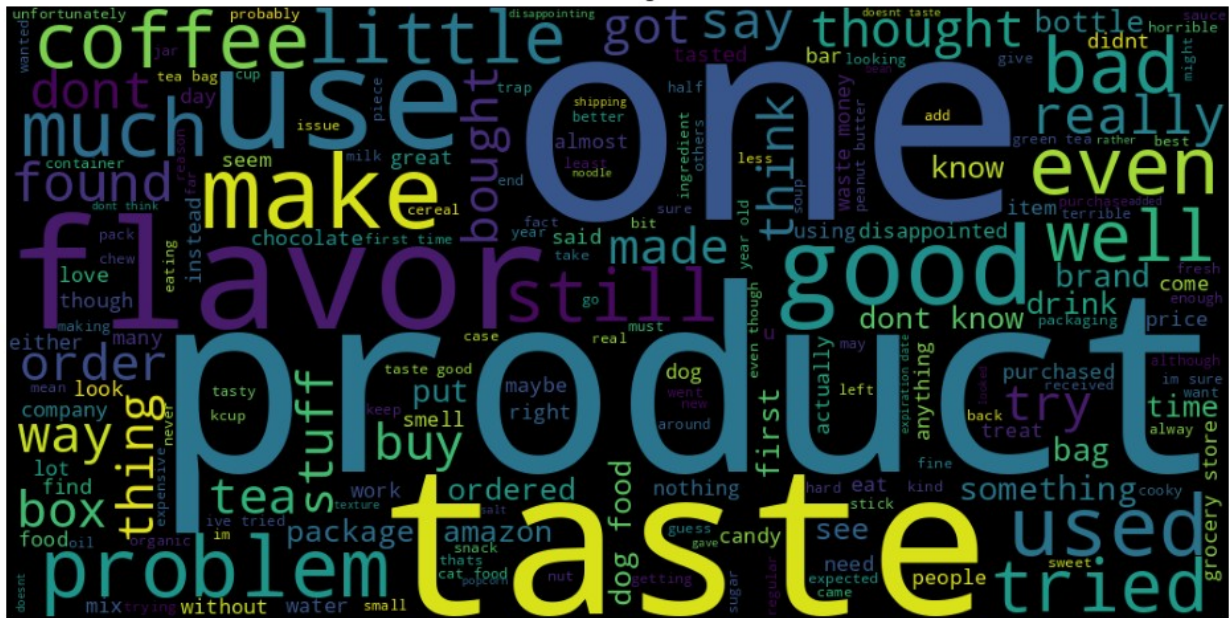
```
VADER_Sentiment
Positive     496975
Negative      59335
Neutral       12144
Name: count, dtype: int64
```

```
from wordcloud import WordCloud
```

```
positive_reviews = " ".join(
    df[df['VADER_Sentiment']=="Positive"]['Cleaned_Text']
)
```



## Most Common Negative Words



## **5. Inform marketing, product development, or social insights**

### **Marketing Insights**

- Most reviews are positive, indicating strong customer satisfaction.
- Positive keywords (e.g., "delicious", "fresh", "quality") can be highlighted in advertising campaigns.

### **Product Development**

- Negative reviews often mention issues like "stale", "broken", "late", or "expired".
- These recurring complaints can guide improvements in packaging, shipping, and quality control.

### **Customer Experience**

- Neutral reviews represent opportunities to improve products or service quality and convert these customers into promoters.

### **Business Recommendations**

Based on the sentiment analysis of Amazon Food Reviews, the following recommendations can help improve business performance and customer satisfaction:

#### **1 )Enhance Product Quality**

- Identify products receiving frequent negative feedback and investigate issues related to taste, freshness, packaging, or quality.
- Implement stricter quality control measures to reduce customer complaints.

## **2 )Improve Customer Experience**

- Respond promptly to negative reviews and address customer concerns.
- Encourage satisfied customers to leave detailed reviews, helping build trust among potential buyers.

## **3 )Optimize Marketing Strategies**

- Use positive customer feedback and commonly praised features in advertising campaigns.
- Highlight strengths such as product quality, flavor, freshness, and value for money in promotional content.

## **4 )Product Development**

- Analyze recurring negative keywords to identify opportunities for product improvement.
- Use customer suggestions to develop new products or enhance existing ones based on consumer preferences.

## **5 )Monitor Customer Sentiment Regularly**

- Perform periodic sentiment analysis to track changes in customer opinions.

- Use sentiment trends as an early warning system for potential product or service issues.

## 5. Feature Extraction using TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    max_features=5000,
    stop_words='english'
)

X = tfidf.fit_transform(df['Cleaned_Text'])

print(X.shape)

(568454, 5000)
```

### Prepare Target Variable

```
y = df['Score']

def sentiment(score):
    if score >= 4:
        return "Positive"
    elif score == 3:
        return "Neutral"
    else:
        return "Negative"

df['Sentiment'] = df['Score'].apply(sentiment)

from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

y = encoder.fit_transform(df['Sentiment'])

print(encoder.classes_)

['Negative' 'Neutral' 'Positive']
```

### Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
    test_size=0.20,  
    random_state=42  
)
```

## Train the Model

```
from sklearn.naive_bayes import MultinomialNB  
  
model = MultinomialNB()  
  
model.fit(X_train, y_train)  
  
MultinomialNB()
```

## Make Predictions

```
y_pred = model.predict(X_test)
```

## Evaluate the Model

```
from sklearn.metrics import accuracy_score, classification_report,  
confusion_matrix
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Accuracy: 0.8178307869576308

```
print(classification_report(y_test, y_pred))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.83      | 0.27   | 0.41     | 16181   |
| 1            | 0.44      | 0.00   | 0.00     | 8485    |
| 2            | 0.82      | 1.00   | 0.90     | 89025   |
| accuracy     |           |        | 0.82     | 113691  |
| macro avg    | 0.70      | 0.42   | 0.44     | 113691  |
| weighted avg | 0.79      | 0.82   | 0.76     | 113691  |

```
cm = confusion_matrix(y_test, y_pred)
```

```
print(cm)
```

```
[[ 4377  14 11790]  
 [  458  16  8011]  
 [  432   6 88587]]
```

## Predict Sentiment for New Reviews

```
review = ["This product tastes amazing and I will definitely buy it  
again."]  
  
review_vector = tfidf.transform(review)  
  
prediction = model.predict(review_vector)  
  
print(encoder.inverse_transform(prediction))  
  
['Positive']  
  
review = ["Worst product ever. Completely disappointed."]
```

# Conclusion

- This project successfully analyzed customer reviews from the Amazon Fine Food Reviews dataset using Natural Language Processing (NLP) techniques. The text data was preprocessed by removing noise, stop words, punctuation, and applying lemmatization. Customer sentiments were then classified into Positive, Negative, and Neutral categories using both machine learning and lexicon-based approaches.
- The analysis provided valuable insights into customer opinions, identified common themes in positive and negative reviews, and highlighted opportunities for business improvement. The findings demonstrate how sentiment analysis can support data-driven decision-making in areas such as product quality, marketing strategy, customer experience, and product development.
- Overall, this project showcases the practical application of NLP and sentiment analysis in transforming unstructured customer feedback into meaningful business insights.

# Thank You!

Thank you for reviewing this project.

This project demonstrates the practical application of Natural Language Processing (NLP) and Sentiment Analysis to extract meaningful insights from customer reviews. It reflects my skills in Python, Data Preprocessing, NLP, Machine Learning, Data Visualization, and Business Analytics.

I appreciate your time and feedback.